CS30401 Network Lab 1: Lab Environment Guide

Read this first. This guide is a starting point, not a model answer. It shows how to install the lab environment, how to use the tools we provide, what you can observe with them, and what you can change. The Seoul–London example below shows the workflow only.

1. The Environment

1.1 What it is

The environment is **ns-3.48** with the LEO satellite modules of **Hypatia** [1], ported to ns-3.48 by the TA. Hypatia models a satellite constellation in ns-3: satellites move along their orbits, ground stations connect to satellites in view, and packets are forwarded along routes that are recomputed as the satellites move. Around it we provide tools to generate constellations, run experiments, and look at the results.

Item	Default	Can you change it?
Constellation	One shell: Starlink shell 1, 72 planes × 22 satellites = 1,584, altitude 550 km, inclination 53°, minimum elevation angle 25° (Hypatia [1], from SpaceX's FCC filing)	Number of satellites: an option. Several shells, each with its own altitude, inclination, and minimum elevation angle: an option. Other orbital parameters: by editing the generator (§4.2)
ISLs	+Grid: 4 per satellite (2 in the same plane, 1 in each adjacent plane)	By editing the generator. With several shells, each shell has its own +Grid and there are no ISLs between shells
Ground stations	Seoul and London	Yes, any number and location (§4.1)
Path	Ground station → satellite → ISLs → satellite → ground station; no ground relays	Fixed for this lab
Routing	Shortest path by distance, recomputed every 100 ms (Hypatia [1])	Update interval: an option (§4.3)
Links	All ISLs and GSLs share one data rate and one drop-tail queue size	Yes, per run (§4.4)
Traffic	Bulk TCP flows between ground stations	Yes (§4.4)

1.2 What it models, and what it does not

Routes are not computed during the simulation. `gen_state.py` computes them in advance from the satellite orbits, for every routing step (every 100 ms by default), and stores them in the state directory (`state/dynamic_state_100ms_for_200s/fstate_<time in ns>.txt`); ns-3 loads the table for each step at that time. `path_truth.py` and `viz_path.py` read the same files.

Know what the simulator does, so that you can decide what your study needs:

- Propagation delay follows the actual distance between nodes at every moment.
- At every routing update (every 100 ms), forwarding tables and each ground station's satellite switch instantly. There is no link-layer handover procedure, no outage, and no bit error.
- Packets are lost only when a device queue overflows, or when a ground station sees no satellite.
- Each link has a drop-tail device queue and no queueing discipline.
- TCP runs with these settings, applied by Hypatia before your flows start (printed at the top of every run): initial cwnd 10 segments, segment size 1380 bytes, send and receive buffers 32 MiB, SACK, timestamps, and window scaling on. Everything else is the ns-3.48 default.

Anything else your study requires, you add yourself and justify, as the handout asks.

1.3 What is included

Tool	What it does
`check_env.sh`	Checks the installation: ping RTT in ns-3 vs. RTT computed from satellite positions (§3.1)
`gen_state.py`	Generates a constellation state: orbits, ISLs, ground stations, and forwarding tables for every routing step (§4)
`make_run.sh`	Creates a run directory: link rate, queue size, TCP variant, flows (§3.2, §4.4)
`run_sim.sh`	Builds any ns-3 changes and runs the simulation on a run directory (§3.2)
`viz_path.py`	Animated globe view of the satellites and the path between two ground stations (§3.4)
`path_truth.py`	The path and its propagation RTT at every routing step, as a CSV file (§3.4)
`make_patch.sh`	Saves every change you made to ns-3 as one patch file for your submission (§6)
`setup_ns3.sh`	Installs the satellite modules into ns-3 and builds it (used by the manual installation)
Example state	Seoul (node 1584) and London (node 1585), 200 s, at `$STATE_EXAMPLE`

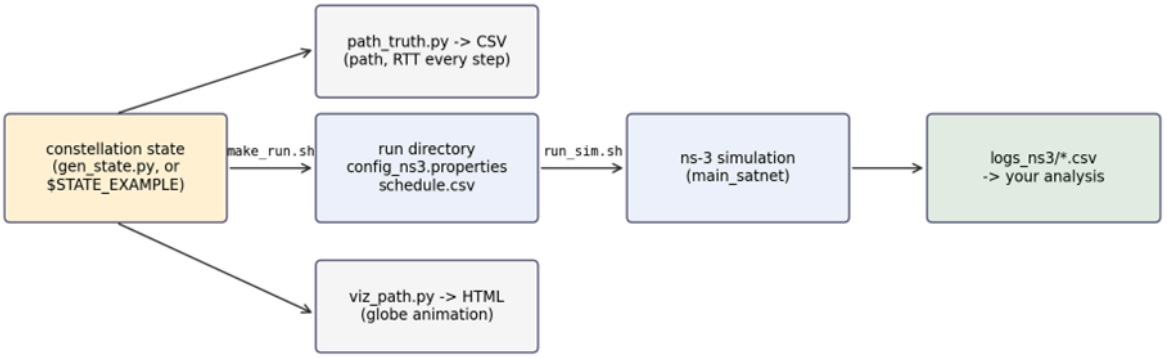


Figure 1. How the tools fit together.

2. Installation

There are two ways to install the environment, and both give the same environment. Whichever you use, state it in your `README.md`: the Docker image tag (for example, `2026f-v1`), or for a manual installation, your OS, compiler version, ns-3 version, and the commit of the environment repository. We reproduce your results in the environment you state.

If you are familiar with these tools, the sources are:

Source	Link
Docker image	`ghcr.io/yeonjong21/cs30401-lab1:2026f-v1` (package page)
Environment repository	https://github.com/Yeonjong21/cs30401-lab1-env
ns-3.48	https://github.com/nsnam/ns-3-dev-git/tree/ns-3.48
Hypatia (only `satgenpy` is used; pinned commit)	https://github.com/snkas/hypatia/tree/0ac531c313eba2335f6344b46347140c3a0d4230

The commands in this guide were tested on Ubuntu 24.04 (Docker image and WSL2). On other systems they may differ slightly, for example in package names, the need for `sudo`, or the shell syntax (PowerShell on Windows).

	Option A: Docker (recommended)	**Option B: Manual installation**
Works on	Windows, macOS (Intel and Apple Silicon), Linux	Ubuntu 22.04 / 24.04, or WSL2 on Windows
Setup time	Download once (about 0.6 GB), no build	About 10 to 30 minutes (building ns-3)
You install	Docker only	ns-3.48, the environment repository, Hypatia's satgenpy, Python packages

2.1 Option A: Docker

Docker in brief

Docker runs a prepared Linux environment on your computer, the same on every OS.

Term	**Meaning in this lab**
image	The packaged environment: Ubuntu 24.04, ns-3.48 (built), the satellite modules, and our tools. You download it once.
container	A running copy of the image. You work inside it. We start it with `--rm`, so it is deleted when you exit.
volume	Storage managed by Docker that outlives containers. Your ns-3 tree lives in the volume `cs30401-ns3`.
bind mount	A folder of your computer made visible inside the container. Your lab folder appears as `/work`.
tag	The version after the colon in the image name (`2026f-v1`). Report it in your README.

Install Docker (once)

- **Windows, macOS**: Docker Desktop. Start Docker Desktop before using `docker` commands. On Windows, if Docker Desktop reports that virtualization is disabled, enable it in the BIOS/UEFI; if it asks you to update WSL, run `wsl --update` in PowerShell.
- **Linux**: Docker Engine. Then run `sudo usermod -aG docker $USER` and log out and back in, so that you can run `docker` without `sudo`.

Check that it works: the following prints "Hello from Docker!".

```
docker run --rm hello-world
```

Download the image (once)

```
docker pull ghcr.io/yeonjong21/cs30401-lab1:2026f-v1
```

The same command works on Intel/AMD and Apple Silicon computers; Docker picks the right version.

Start the environment

Make a folder for your lab work, open a terminal **in that folder**, and run:

macOS, Linux, WSL:

```
docker run -it --rm --name lab1 \
  -v cs30401-ns3:/home/ubuntu/ns-3.48 \
  -v "$PWD":/work \
  ghcr.io/yeonjong21/cs30401-lab1:2026f-v1
```

Windows PowerShell (one line):

```
docker run -it --rm --name lab1 -v cs30401-ns3:/home/ubuntu/ns-3.48 -v
"${PWD}:/work" ghcr.io/yeonjong21/cs30401-lab1:2026f-v1
```

Option	Meaning
`-it`	Interactive terminal: you get a shell inside the container.
`--rm`	Delete the container on exit. Your data is in `/work` and in the volume, not in the container.
`--name lab1`	Name the container, so that other commands can refer to it.
`-v cs30401-ns3:/home/ubuntu/ns-3.48`	Keep ns-3 (your edits and builds) in the volume `cs30401-ns3`.
`-v "$PWD":/work`	Show the current folder of your computer as `/work`.

A welcome message appears and the prompt changes. You are now in a Linux shell inside the container. Type `exit` to leave.

Where am I?

You now have two places to type commands, and the same folder appears in both under different names. The prompt tells you where you are (Figure 2):

- `you@laptop:~/lab1$` is **your computer**. Docker commands, opening files in a browser, and your own editor run here.
- `ubuntu@3f2a...:/work$` is **inside the container**. ns-3 and all our tools run here.

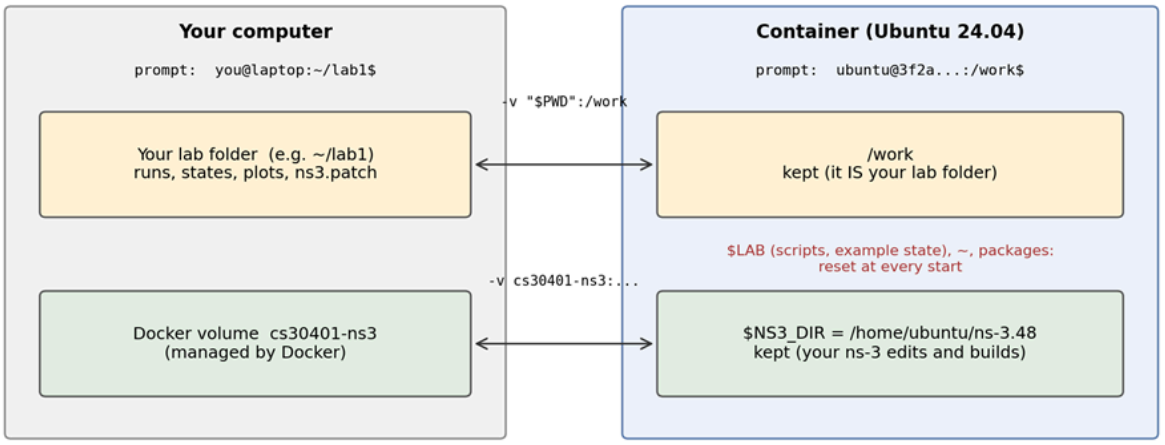


Figure 2. What lives where. Only `/work` and `$NS3_DIR` are kept when you exit.

Location in the container	Kept on exit?	
`/work`	Yes	This is your lab folder on your computer. Keep code, states, runs, and plots here.
`$NS3_DIR` (`/home/ubuntu/ns-3.48`)	Yes	Stored in the volume `cs30401-ns3`. Your changes to ns-3 and its build survive.
Everything else (`$LAB`, `~`, installed packages)	No	Reset every time the container starts. Copy a script to `/work` before you edit it.

Other Docker commands you may need, run on **your computer**:

Command	What it does
`docker exec -it lab1 bash`	Open a second terminal in the running container.
`docker ps`	List running containers.
`docker stop lab1`	Stop the container (for example, if its terminal was closed).
`docker volume rm cs30401-ns3`	Delete your ns-3 volume; the next start gives a clean ns-3. Save a patch first (§6).
`docker system df`	Disk space used by Docker.

2.2 Option B: Manual installation

Tested on Ubuntu 24.04 and 22.04, including inside WSL2. On WSL2, work under your Linux home directory (`~`), not under `/mnt/c/`. You download three things:

What	Why
ns-3.48	The simulator.
The environment repository	The satellite modules ported to ns-3.48, our tools, and the example state. Hypatia's own ns-3 modules are for ns-3.31 and do not build on ns-3.48.
Hypatia	Only its `satgenpy` package is used, to compute satellite positions and routes.

```
# 1. System packages
sudo apt update
sudo apt install -y g++ python3 python3-venv git curl bzip2 cmake ninja-build

# 2. uv (Python package manager); open a new terminal afterwards
curl -LsSf https://astral.sh/uv/install.sh | sh

# 3. Ubuntu 22.04 only: its cmake (3.22) is too old for ns-3.48 (needs 3.25+)
uv tool install cmake && cmake --version

# 4. The environment repository, and Hypatia (pinned version) inside it
mkdir -p ~/cs30401 && cd ~/cs30401
git clone https://github.com/Yeonjong21/cs30401-lab1-env.git lab
git clone https://github.com/snkas/hypatia.git lab/hypatia
git -C lab/hypatia checkout 0ac531c313eba2335f6344b46347140c3a0d4230

# 5. ns-3.48 (the same source as the Docker image)
curl -L -o ns-3.48.tar.gz
https://codeload.github.com/nsnam/ns-3-dev-git/tar.gz/refs/tags/ns-3.48
tar xzf ns-3.48.tar.gz && mv ns-3-dev-git-ns-3.48 ns-3.48

# 6. Install the satellite modules into ns-3 and build (10 to 30 min)
bash lab/scripts/setup_ns3.sh ~/cs30401/ns-3.48

# 7. Python packages
cd ~/cs30401/lab && uv sync --frozen
```

Then add these lines to `~/.bashrc` and open a new terminal. They set up what the Docker image sets up for you:

```
export NS3_DIR=~/cs30401/ns-3.48
export LAB=~/cs30401/lab
export STATE_EXAMPLE=$LAB/data/seoul_london/state
export PATH=$LAB/scripts:$PATH
source $LAB/.venv/bin/activate
```

`setup_ns3.sh` does exactly what the Docker image build does: it copies the satellite modules into ns-3, records a git baseline for your patch (§6), and builds with `--build-profile=release` (optimized, but without `-march=native`, so results do not depend on the CPU that compiled them). In the rest of this guide, replace `/work` with a folder of your own.

2.3 Opening result files on your computer

Results written to `/work` appear in your lab folder. To open it in your file browser, run on **your computer**: `explorer.exe .` in WSL (Windows), `open .` on macOS, `xdg-open .` on Linux. On Windows, you can also browse WSL folders by typing `\\wsl$` in the address bar of the File Explorer.

3. Welcome Example: Seoul to London

This section walks you through a toy example, sending TCP traffic from Seoul to London, to help you get familiar with the environment: checking the installation, running a flow, reading the outputs, and seeing the path. It shows how to use the tools.

All commands in this section run inside the container (or, for a manual installation, in a terminal with the `~/.bashrc` lines above).

3.1 Check the environment

```
check_env.sh
```

It sends pings from Seoul to London for 100 simulated seconds and compares their RTT with the propagation RTT computed independently from satellite positions. It takes one to two minutes. The last line may read:

```
pings 9992   ping RTT - truth [ms]: min -2.580  median 0.672  p99 0.805  max 0.805
```

The median difference of 0.67 ms is the transmission delay of the ping packets on each hop, so ns-3 reproduces the geometric RTT.

3.2 Run one TCP flow

```
make_run.sh /work/runs/r1 $STATE_EXAMPLE TcpCubic 100 1000 20 1584 1585
run_sim.sh  /work/runs/r1
cat /work/runs/r1/logs_ns3/tcp_flows.txt
```

Argument	Meaning
`/work/runs/r1`	Run directory to create
`$STATE_EXAMPLE`	Constellation state (here the example; see §4 for your own)
`TcpCubic`	TCP variant, by its ns-3 class name (§4.4)
`100`	Data rate of every ISL and GSL, Mbit/s
`1000`	Queue size of every ISL and GSL device, packets
`20`	Simulated time, seconds (at most the duration of the state)
`1584 1585`	Sender and receiver ground stations (node ids from `state/node_ids.txt`)

The run takes about a minute. `tcp_flows.txt` summarizes each flow; here the flow reaches an average of 92.6 Mbit/s. The simulation is deterministic, so you get exactly the same numbers.

```
TCP Flow ID  Source  Target  ...  Sent          Progress  Avg. rate     Finished?
0            1584    1585    ...  1851.02 Mbit  0.2%      92.6 Mbit/s  NO_ONGOING
```

`NO_ONGOING` means the flow was still sending when the simulation ended: `make_run.sh` creates bulk flows of 100 GB that never finish.

3.3 Look at the output files

File in `logs_ns3/`	Columns
`tcp_flow_<id>_cwnd.csv`	flow id, time (ns), cwnd (bytes); one line per change
`tcp_flow_<id>_rtt.csv`	flow id, time (ns), smoothed RTT (ns); one line per change
`tcp_flow_<id>_progress.csv`	flow id, time (ns), bytes acknowledged so far
`tcp_flows.csv`, `tcp_flows.txt`	Per-flow summary

Analysis is yours to write. For example, a few lines of Python plot the congestion window (Python and matplotlib are available in the container):

```
import csv, matplotlib.pyplot as plt
rows = csv.reader(open("/work/runs/r1/logs_ns3/tcp_flow_0_cwnd.csv"))
t, cwnd = zip(*[(int(r[1]) / 1e9, int(r[2]) / 1380) for r in rows])
plt.step(t, cwnd, where="post")
plt.xlabel("simulated time (s)"); plt.ylabel("cwnd (segments)")
plt.savefig("/work/cwnd.png")
```

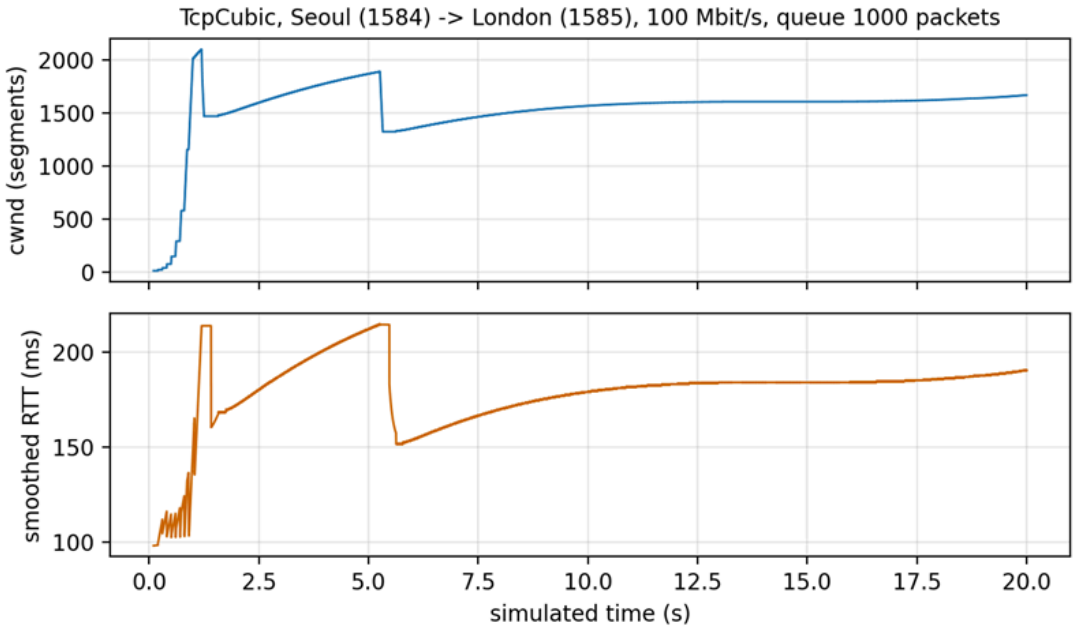


Figure 3. cwnd and smoothed RTT of the flow in §3.2.

3.4 See the path

```
viz_path.py $STATE_EXAMPLE 1584 1585 --from 20 --to 30
```

This writes `/work/viz/path_1584_1585_20-30s.html` (about 30 seconds). Open it in a web browser on your computer (§2.3). It shows a globe with every satellite, the two ground stations, and the path between them over time. Play/Pause and the time slider are at the bottom; drag to rotate the globe and scroll to zoom. The box at the top left shows the time, the hop count, the propagation RTT, and the times at which the path changed. It needs an internet connection to load the plotting library; no account is needed.



Figure 3. The globe view of LEO satellite network from Seoul to London

The same information as a table, one row per routing step:

```
python3 $LAB/scripts/path_truth.py $STATE_EXAMPLE dynamic_state_100ms_for_200s 1584
1585 /work/path.csv
```

Columns: time (s), hop counts of the forward and reverse paths, propagation RTT (ms), whether the path changed at that step, and the forward path as a list of node ids.

3.5 Where to go from here

A few directions to start exploring. None of them is the answer; they are ways to look.

- Run the common congestion control algorithms (NewReno, CUBIC, BBR, Vegas) on the same scenario and compare how they behave over time, not only their average throughput.
- Vary the conditions: link rate, queue size, simulated time, ground station pairs, the number of users sharing the network.
- When something in the TCP logs changes, check what the network was doing at that moment (§3.4).
- Ask whether what you see is caused by the LEO environment: would it also happen on a path whose properties do not change?

- Add the observations you need. ns-3 exposes many internal signals as *trace sources* you can connect to in the scenario program, for example the congestion window, RTT estimates, TCP's congestion state, transmitted and received packets, and queue drops (see the ns-3 manual on tracing [2]).

4. Configuring the Environment

A constellation state is generated once and then used by any number of runs. The example state covers Seoul and London for 200 s. For anything else, generate your own with `gen_state.py`:

```
python3 $LAB/scripts/gen_state.py /work/mystate --threads 4 --duration 200 \
      --gs   "Seoul,37.56826,126.97783"   --gs   "London,51.50853,-0.12574"   --gs
"Tokyo,35.6895,139.69171"
```

The state is written to `/work/mystate/state/`. Use that path wherever `$STATE_EXAMPLE` appears in §3. Run `python3 $LAB/scripts/gen_state.py --help` for all options.

4.1 Ground stations

- Give each ground station as `--gs "name,latitude,longitude"` in decimal degrees (north and east positive, south and west negative). The name is only a label.
- For many stations, write them in a file, one `name,lat,lon` per line (lines starting with `#` are ignored), and pass it with `--gs-file`:

```
cat > /work/stations.txt << EOF
Seoul,37.56826,126.97783
London,51.50853,-0.12574
Tokyo,35.6895,139.69171
EOF
python3   $LAB/scripts/gen_state.py   /work/mystate   --threads   4   --gs-file
/work/stations.txt
```

- Ground station node ids follow the satellites, in the order given (1584, 1585, … by default). Always check `state/node_ids.txt`.
- **Coordinates need a basis too.** Hypatia's list of the 1,000 most populous cities is in the Docker image; the Seoul and London defaults come from it:

```
grep -i -E ",(Tokyo|Paris),"
$SATGENPY/../paper/satellite_networks_state/input_data/ground_stations_citie
s_sorted_by_estimated_2025_pop_top_1000.basic.txt
```

- With the default shell (inclination 53°, minimum elevation 25°), stations above roughly 61° of latitude rarely or never see a satellite.

4.2 Satellites and shells

Satellites are added shell by shell, not one at a time. Without any option, `gen_state.py` builds the default shell below. It is an idealized constellation computed from the design values in SpaceX's FCC filing, as in Hypatia [1], not the positions of real satellites.

Value	Default shell (Starlink shell 1)
Orbital planes	72, evenly spaced (5° apart)
Satellites per plane	22, evenly spaced; neighboring planes are phase-shifted
Altitude and orbit	550 km, circular (mean motion 15.19 revolutions per day)
Inclination	53°
Minimum elevation angle	25° (a ground station reaches satellites up to about 1,090 km away)
ISLs	+Grid: 2 in the same plane, 1 in each adjacent plane

Change the number of satellites in the default shell:

```
python3 $LAB/scripts/gen_state.py /work/mystate --threads 4 --orbits 72
--sats-per-orbit 30
```

Use several shells: give one `--shell "altitude_km,inclination_deg,orbits,sats_per_orbit,min_elevation_deg"` per shell. `--shell starlink1` is the default shell above.

```
python3 $LAB/scripts/gen_state.py /work/mystate --threads 4 \
      --shell starlink1 --shell "540,53.2,72,22,25"
```

- Satellites are numbered shell by shell; the output lists the id range of each shell. Each shell has its own +Grid ISLs and there are no ISLs between shells, so a path stays within one shell.
- **Where to find values:** SpaceX's FCC filings (altitude, inclination, planes, and satellites per plane of each Starlink shell); the Hypatia paper [1] and Bhattacherjee and Singla [4], which also describe other constellations. CelesTrak publishes the orbits of real satellites, but using them requires changing the generator.

Anything else (for example, a single satellite in a special orbit) requires editing a copy of `gen_state.py`.

4.3 Time span and routing steps

- `--duration` (default 200 s) is the time span of the state; a run cannot simulate longer than that.
- `--step-ms` (default 100) is the routing update interval.
- `--threads` splits the work over CPU cores. 200 s of the default state takes about 25 minutes on one thread.
- Path changes can be tens of seconds apart; generate a long state and choose windows within it.

4.4 Links, TCP, and traffic

`make_run.sh` writes two plain text files into the run directory. Edit them for anything it does not cover.

- **TCP variant:** any ns-3 congestion control class name, such as `TcpNewReno`, `TcpCubic`, `TcpBbr`, or `TcpVegas`. All flows in a run use the same one.
- **Several flows:** set `FLOWS` instead of the last two arguments. Each `a>b` is one TCP connection in which a sends data to b; list both directions to get both.

```
FLOWS="1584>1585,1586>1585" make_run.sh /work/runs/two /work/mystate/state
TcpCubic 100 1000 60 0 0
```

- **Several users at one ground station:** repeat the same pair. Each entry is a separate TCP connection, and they share that ground station's link to its satellite, like several users behind one terminal or in one area. To give users links of their own, add them as separate ground stations at nearby locations instead.

```
FLOWS="1584>1585,1584>1585,1584>1585" make_run.sh /work/runs/users3
$STATE_EXAMPLE TcpCubic 100 1000 60 0 0
```

- `schedule.csv`: one line per flow (`id,from,to,size_bytes,start_ns,,`); change sizes and start times here.
- `config_ns3.properties`: link rates, queue sizes (ISLs and GSLs separately), TCP variant, simulated time, and seed. The format is documented in `$NS3_DIR/contrib/basic-sim/doc/`.

5. What You Can Change

You may change anything: the scenario, TCP, the satellite modules, the state generator. The table is a map of where things live, not a list of solutions.

Direction	Examples	Where
Adjust TCP parameters	Initial congestion window, duplicate-ACK threshold, buffer sizes, with `Config::SetDefault(...)`	`scratch/main_satnet/main_satnet.cc`
Use existing algorithms	Congestion control: `TcpNewReno`, `TcpCubic`, `TcpVegas`, `TcpBbr`, … Loss recovery: `TcpClassicRecovery`, `TcpPrrRecovery`	Run configuration, or the scenario program
Write a new congestion control algorithm	A class derived from `TcpNewReno` or `TcpCongestionOps`	New files in `scratch/main_satnet/`
Change how TCP detects and recovers from loss	What TCP treats as a loss, and what it does then	`src/internet/model/tcp-socket-base.cc`

Use knowledge of the LEO network	Future route changes are known in advance	The scenario program
Act as the network operator	How satellites or ground stations forward packets	`contrib/satellite-network/`
Observe more	ns-3 trace sources and your own logs	The scenario program

After editing ns-3, `run_sim.sh` rebuilds automatically. Python packages installed inside the container are lost on exit; keep them in a `pyproject.toml` in `/work` and run `uv sync` after each start.

6. Saving and Reproducing Your Work

6.1 Your ns-3 changes as a patch

`make_patch.sh` writes every change and every new file in the ns-3 tree to `/work/ns3.patch`. To apply it to a clean ns-3:

```
cd $NS3_DIR && git apply /work/ns3.patch && ./ns3 build
```

Test this before you submit: start the image with a new, empty volume (for example `-v cs30401-ns3-clean:/home/ubuntu/ns-3.48`), apply your patch, and run your `run_all.sh`.

7. Troubleshooting

Symptom	Fix
`command not found` for our tools or for `explorer.exe`	You are in the wrong place: our tools run inside the container, `explorer.exe` and Docker commands on your computer. Check the prompt (§2.1).
`permission denied ... docker.sock` (Linux)	`sudo usermod -aG docker $USER`, then log out and back in.
`permission denied` on `/work` (Linux)	If `id -u` on your computer is not 1000, run `chmod a+rwx` on your lab folder.
Cannot connect to the Docker daemon	Start Docker Desktop, or `sudo systemctl start docker` on Linux.
A build or run is killed without a message	Out of memory. Give Docker (or WSL) more memory; see §4.2 for larger constellations.
`sim_s ... exceeds the state duration`	Generate a longer state (`--duration`) or simulate less.

The globe view is empty	It needs an internet connection.
`cmake` version error (manual installation)	Ubuntu 22.04: step 3 of §2.2.

If you are stuck, post on Classum with the “Lab, Q&A” tag, including your OS, the exact command, and the full error message.

References

1. S. Kassing, D. Bhattacherjee, A. B. Aguas, J. E. Saethre, and A. Singla, “Exploring the ‘Internet from space’ with Hypatia,” *Proceedings of the ACM Internet Measurement Conference (IMC)*, 2020. Code: https://github.com/snkas/hypatia
2. ns-3 documentation
   - Tutorial (Conceptual Overview, Tracing): https://www.nsnam.org/docs/tutorial/html/
   - TCP models: https://www.nsnam.org/docs/models/html/tcp.html
   - API: https://www.nsnam.org/docs/release/3.48/doxygen/
3. Environment repository (source of the Docker image): https://github.com/Yeonjong21/cs30401-lab1-env
4. D. Bhattacherjee and A. Singla, “Network Topology Design at 27,000 km/hour,” *Proceedings of the ACM Conference on Emerging Networking Experiments and Technologies (CoNEXT)*, 2019.